

Experiment - 03.

* AIM: Implement Min, max, Sum and average operations using parallel reduction.

* THEORY: Parallel Reduction.

(1). Parallel reduction - is a technique used in parallel computing to reduce the results of an operation on a set of input data into a single value. In this case, we will be discussing implementation of Min, max, Sum and average operations using parallel reduction.

(2) To implement these operations using parallel reduction, we need to follow these steps -

1. Divide the input data in equal sized chunks among the available threads.
2. Calculate the local Min, max, sum and average for each thread on their respective chunks.
3. Combine the local Min, max, sum and average values across all threads using the parallel reduction technique.
4. Finally obtain the final min, max, sum, average values by combining the results of all threads.

(3) Here's how we can implement each of these operations.

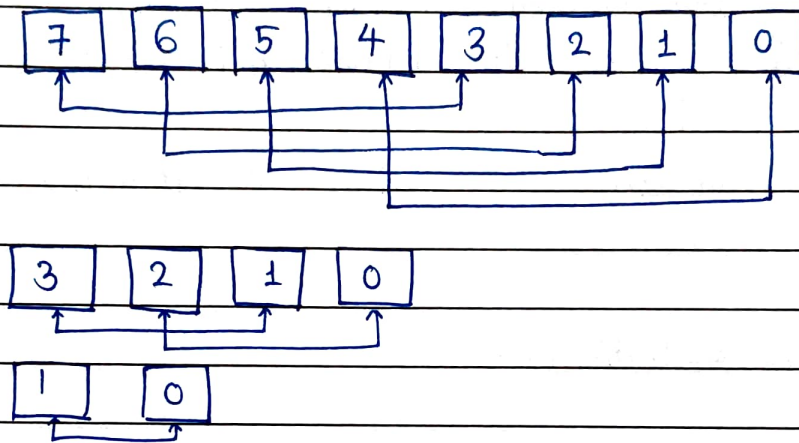
(a) Min operation - To perform min operation using parallel reduction we can start by initializing a variable to a very large value (eg. INT_MAX for integer data). Each thread then calculates the local minimum for its chunk of data and the global minimum is found by comparing local minimum values from each thread and taking the smallest one.

(b) Max Operation - Similar to the min, we can initialize a variable to a very small value (eg INT_MIN for integer data) to perform a max operation. Each thread calculates the local maximum for its chunk of data, and the global maximum is found by comparing the local maximum values from each thread and taking the largest one.

(c). Sum operation - To perform sum operation using parallel reduction, each thread calculates the local sum for its chunk of data, and the global maximum is found by comparing to local maximum values from each thread and taking the largest one. , obtained by summing up local ~~sum~~ values from each thread.

(d). Average Operation -

To perform average operation using parallel reduction, we first perform a sum operation on input data. Then we divide the global sum by the total number of elements in the input data to obtain global average.

(4). Void Min-reduction()

- Declares a void function that takes a reference to an integer vector as its argument.
- `int min-value = INT_MAX`, initializes an integer variable `min-value` to largest possible integer value using the `INT_MAX` constant from the `<climits>` header file. This is done to ensure that `min-value` is initially greater than any element in `arr`.
- `#pragma omp parallel for reduction (min : min-value)` is an Open MP directive that specifies that the following loop should be executed in parallel using multiple threads.
- The `reduction (min : min-value)` clause indicates that each thread should be maintaining a private copy of `min-value` and update it with minimum value it finds.

in its portion of loop. Once the loop is complete, OpenMP will combine all the private copies of min-value into a single shared value that represents the minimum value in Arr.

- 'for' loop iterates over each element of arr.
- `if (arr[i] < min-value) { min-value = arr[i]; }` checks if the current element of arr is less than min-value. If so, it updates min-value to be the current element.
- `cout << "Min element";` prints out minimum value found in arr.

(5). void max-reduction() -

- It declares a void function that takes a reference to an integer vector as its argument.
- `int max-value = INT_MIN` initializes an integer variable max-value to the smallest possible integer value using INT_MIN constant from the `<limits>` header file. This is done to ensure that max value is initially smaller than any element in arr.
- `#pragma omp parallel for reduction (max: max value)` is an OpenMP directive that specifies that the following loop should be executed in parallel using multiple threads. The reduction clause indicates that each thread should maintain a private copy of max-value and update it with the maximum value it finds in its portion of the loop.
- Once the loop is complete, OpenMP will continue all

private copies of max-value into a single shared value that represents the maximum value in arr.

- 'for' loop iterates over each element of arr.
- if (arr[i] > max-value) { max-value = arr[i] } checks if the current element of arr is greater than max-value. If so, it updates max-value to be the current element.
- cout << "Maximum value:" << max-value << endl; prints out the maximum value found in arr.

(6). #include <climits> -

- <climits> is a header file in c++ that contains constants related to integer types. This header file provides implementation - defined constants for minimum and maximum values of integral types, such as int-max and int-min.
- Using these constants instead of hard coding the values of the minimum and maximum integer values is a good practice because it makes the code more readable and avoids the possibility of introducing errors in the code.
- The use of these constants also ensures that the code will work correctly across different platforms and compilers.

(7) CUDA -

- Cuda programming is especially well suited to address problems that can be expressed as data parallel computations.
- Data parallel processing maps data elements to parallel elements and threads.
- The first step in designing a data parallel program is to partition data across threads, with each thread working on a portion of data.
- Host code runs on CPUs and device code runs on GPUs. An application executing on heterogeneous platform is typically initialized by the CPU.

* Conclusion -

Thus we successfully implemented min, max, sum, average operations using parallel reduction.

* * * * *